

Lecture 3: The Basics

Laboratory: Coding - Accounting, Finance and Business Consulting

A Lecture With Mixed Content

- The objective of this lecture is to speed up your proficiency in Python by:
- Presenting **Shortcuts, Tips and Tricks**:
 - Present some crucial tools (both known and lesser known) and how to use them conveniently
 - Example: help and coding resources, markdown, debugging
- Illustrating **Common Pitfalls**:
 - Recognize and avoid common mistakes that arise when coding
 - Aliasing/cloning, scoping/namespaces
- Providing useful **Best Practices**:
 - How to structure the folders of a data analysis project, documentation and commenting.
 - Utilizing Python's robust libraries effectively.

Launching the Jupyter Notebook

- **Jupyter Notebook**: A browser-based interface to IPython with dynamic display capabilities.
- **Features**: Executes Python/IPython, includes formatted text, visualizations, equations, widgets, etc.
- You'll view and edit Jupyter notebooks through your browser
 - We need to connect the browser to Python so that we can execute code
- On Anaconda navigator you can simply launch jupyter (remember to select the right environment)
- A terminal should open and start showing what it is doing, something like this:

```
Last login: Thu Sep 28 08:34:43 on console
/Users/dgdi/anaconda3/envs/codepy_slides/bin/jupyter_mac.command
; exit;
(base) dgdi@aimecco ~ %
/Users/dgdi/anaconda3/envs/codepy_slides/bin/jupyter_mac.command
; exit;
```

```

| | | | _ _ _ | _ _ | _ _
| | | | ' _ \ / _ \ / _ \
\ _ \ / | . _ \ / _ \ / _ \
| _ |

```

Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions.

https://jupyter-notebook.readthedocs.io/en/latest/migrate_to_notebook7.html

Please note that updating to Notebook 7 might break some of your extensions.

```

[I 17:09:45.961 NotebookApp] Serving notebooks from local
directory: /Users/dgdi
[I 17:09:45.961 NotebookApp] Jupyter Notebook 6.5.4 is
running at:
[I 17:09:45.961 NotebookApp] http://localhost:8888/?
token=544408c56edce5b33fc468e83cee544cdaf9eb896672384a
[I 17:09:45.961 NotebookApp] or http://127.0.0.1:8888/?
token=544408c56edce5b33fc468e83cee544cdaf9eb896672384a
[I 17:09:45.961 NotebookApp] Use Control-C to stop this
server and shut down all kernels (twice to skip
confirmation).
[C 17:09:45.964 NotebookApp]

```

To access the notebook, open this file in a browser:
<file:///Users/dgdi/Library/Jupyter/runtime/nbserver-7327-open.html>

Or copy and paste one of these URLs:
<http://localhost:8888/?token=544408c56edce5b33fc468e83cee544cdaf9eb896672384a>
or <http://127.0.0.1:8888/?token=544408c56edce5b33fc468e83cee544cdaf9eb896672384a>

Launching the Jupyter Notebook - continued

Your default browser should automatically open and show you the local directory

- The exact address will depend on your system. If the browser does not open automatically
 - Open a browser window (chrome, firefox, edge, etc) window and manually copy one of the url at the bottom of the terminal output
 - In the example above is <http://localhost:8888/?token=544408c56edce5b33fc468e83cee544cdaf9eb896672384a>

Notebooks: Combining Programming and Markup Languages

- A programming language is a formal language used to specify a set of instructions for a computer to execute
 - Python is a programming language
- A Markup language is a system for annotating a document to indicate its structure or presentation
 - HTML, markdown and latex are markup languages
- Using a Notebook we can combine both languages in a single document
 - This is a text cell, we can write text here using markdown
 - The one below is a code cell, we can write code here using Python

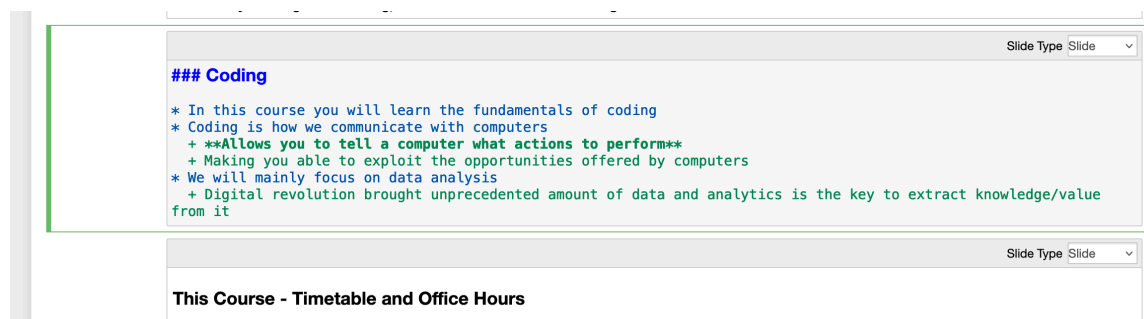
In [1]: `2+2+3+3+2`

Out[1]: 12

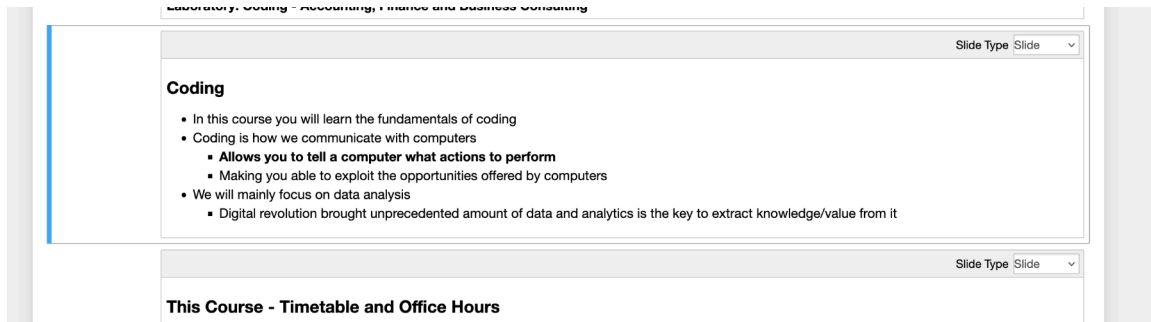
Jupyter notebooks - Modes

- There are two modes in Jupyter notebooks and shortcuts differ among them
- **Edit mode (EM)**: The cursor is inside a cell
 - Allows you to type into a cell and enter text
- **Command mode (CM)**: a cell is selected (but there is no cursor inside it)
 - Allows you to edit the notebook cell-wise
- You can switch between modes using the **Esc** and **Enter** (or **Enter+Shift**) keys
- A convenient way to interact with Jupyter notebooks is keyboard shortcuts
- You can access the list of shortcuts by clicking on the keyboard icon in the toolbar

Edit Mode



Command Mode



Keyboard Shortcuts in Jupyter notebooks - Modes

- Familiar shortcuts (EM): **Copy** Cmd-c/Ctrl-c, **Cut** Cmd-x/Ctrl-x, **Paste** Cmd-v/Ctrl-v, **Undo** Cmd-z/Ctrl-z
 - In command mode: 1.Cmd/Ctrl not needed and, 2. applied to the whole cell
- Other useful shortcuts:
 - Change cell type (CM)** y (code), m (markdown)
 - Run cell (EM and CM)** Cmd-Enter/Ctrl-Enter
 - interrupt evaluation (CM)** i
 - Split cell at cursor (EM)** Ctrl + Shift + -/Ctrl + Shift + =
 - Insert cell above (CM)** a
 - Insert cell below (CM)** b
 - Delete cell (CM)** double d
 - Undo cell deletion (CM)** z
 - Save and checkpoint (EM and CM)** Cmd s/Ctrl s

Running Cells

- The output of a code cell is displayed below the cell
- The output of a markdown cell is displayed as formatted text in-place
- You can run all cells in a notebook by clicking on **Cell** in the menu and then **Run All**
 - Useful to run all cells after you have modified the code
 - To be done only after you have tested the code in all the cells in the notebook since it will stop if an error is raised
- You can also restart the kernel and run all cells by clicking on **Kernel** in the menu and then **Restart & Run All**
 - Useful if you want to start from a clean slate (get rid of all the "boxes" currently cluttering the environment)

Markdown

Markup vs. Programming Languages

| | Markup Languages | Programming Languages

| :----- | :----- | :----- | |

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE recipe PUBLIC "-//Happy-Monkey//DTD RecipeBook//EN"
"http://www.happy-monkey.net/recipebook/recipebook.dtd">
```

```
<recipe>
```

```
  <title>Peanut-butter On A Spoon</title>
```

```
  <ingredientlist>
```

```
    <ingredient>Peanut-butter</ingredient>
```

```
  </ingredientlist>
```

```
  <preparation>
```

```
    Stick a spoon in a jar of peanut-butter,
    scoop and pull out a big glob of peanut-butter.
```

```
  </preparation>
```

```
</recipe>
```

```
def add5(x):
    return x+5
```

```
def dotwrite(ast):
    nodename = getNodeName()
    label=symbol.sym_name.get(int(ast[0]),ast[0])
    print '    %s [label="%s" % (nodename, label),
    if isinstance(ast[1], str):
        if ast[1].strip():
            print '= %s";' % ast[1]
        else:
            print '"'
    else:
        print '";'
        children = []
        for n, child in enumerate(ast[1:]):
            children.append(dotwrite(child))
        print '    %s -> {' % nodename,
        for name in children:
            print '%s' % name,
```

| Examples | TeX, HTML,

XML, **Markdown** | C, Java, JavaScript, **Python**, R

| **Use** | Structure and present data | Transform and generate data

| **Execution** | Program (e.g. a browser) | Computer hardware (Image sources: Wikimedia)

- Here we will briefly explore the markdown language used in text cells of Notebooks
 - We will spend most of our time in the course studying Python

Markdown - Headers

- An R Markdown document is written in markdown (an easy-to-write plain text format) and contains chunks of embedded R code
- You can add headers in your text by using `#`

This is a header

This is a subheader

This is a subsubheader

Markdown - Text Formatting

- for *italic* text, surround the text by underscores or asterisks
 - **Bold** text needs double asterisks just before and after it
 - To add a hyperlink, use `[text](link)` , such as [dSEA website](#)
 - To embed a figure, use `![alternative text](path to the figure)`
 - To add an horizontal line, use `---` at the beginning of a line to get
-

Markdown - Code

- To denote a word or phrase as code, enclose it in backticks ``` , So using ``code`` will produce `code`
- You can also use three backticks to create a code block
- This is useful to show code that you don't want to execute
- You can also specify the language to get some syntax highlighting
- For example, this is a code block in python:

```
if temperature>30
    print("It's hot!")
```

A Markdown Document - Blockquotes

You can also add blockquotes by using `>`

I do believe in simplicity. It is astonishing as well as sad, how many trivial affairs even the wisest thinks he must attend to in a day; how singular an affair he thinks he must omit. When the mathematician would solve a difficult problem, he first frees the equation of all incumbrances, and reduces it to its simplest terms. So simplify the problem of life, distinguish the necessary and the real. Probe the earth to see where your main roots run.

--- Henry David Thoreau

Markdown - Vertical Spacing

- If you want to go to a newline you have to add two spaces at the end of the line and go to a new line in the markdown code

This is a line without spaces at the end- This line has two spaces at the end-
And this is a third line-

- Otherwise you can use a blank line in the markdown code:

This is a line- This line has a blank line below-

And this is a third line-

Markdown - Bullet Point Lists

- You can create bullet point lists by using `*`, `-` or `+` at the beginning of each item
- You can also create nested lists by indenting the items with 2 spaces

-
- item 1
 - item 2
 - item 3

-
- item 1
 - item 1.1
 - item 1.2

Markdown - Numeric and Nested Lists

- You can create numeric lists by using numbers followed by a dot
- You can also create nested numeric nested lists by indenting the items with 3 spaces

-
1. item 1
 2. item 2
 3. item 3

-
1. item 1
 - A. item 1.1
 - B. item 1.1

An R Markdown Document - Math Expressions

- In case you want to add a math expression, you can do it by using LaTeX math syntax
- For inline math expressions you can use the `$` sign, `$NPV= \sum_{t=0}^T \frac{CF_t}{(1+r)^t}` that gives $NPV = \sum_{t=0}^T \frac{CF_t}{(1+r)^t}$
- Using the double dollar `$$` moves the expression into a new line and centers it so that `$$NPV = \sum_{t=0}^T \frac{CF_t}{(1+r)^t}$$` leads to

$$NPV = \sum_{t=0}^T \frac{CF_t}{(1+r)^t}$$

- A good practice to avoid markdown rendering mistakes is to leave at least one empty line between different elements that are adjacent

Using HTML in Markdown

- Using HTML tags you can do many things in markdown, for example:
 - Make superscripts by using `<sup> </sup>` tags: `x<sup>2</sup>` leads to x^2
 - Make subscripts by using `<sub> </sub>` tags: `x<sub>2</sub>` leads to x_2
 - Change the color of the text: `<font color='red'> red text </font>` leads to red text
 - Change the size of the text: `<font size='10'> big text </font>` leads to big text
 - Change the font: `<font face='courier'> courier text </font>` leads to courier text
 - Change the background color: `<span style="background-color: #FFFF00"> yellow background </span>` leads to yellow background
 - Change the font color: `<span style="color: #FFFF00"> yellow text </span>` leads to yellow text

Using HTML in Markdown - Images

- Sometimes markdown it is not properly rendered
 - Lots of different markdown dialects interpreted different by different programs
 - Especially true for images and the options to modify them (dimensions, position, etc)
 - It is preferable to use html
- `` leads to



Also markdown has its own syntax to handle image size, but it is not always rendered correctly, i.e., `![alternative text](img/pylogo.png =200x200)` leads to

`![alternative text](img/pylogo.png =200x200)`

Python Language Basics

- Overview of Python programming basics
 - Essential language mechanics
- Tips and tricks to make your coding life easier

Objects, attributes and methods

- Python is object-oriented, (almost) everything is an object
 - Integers, lists, functions, libraries, etc.

Objects, attributes and methods

- Objects (usually) have attributes and methods
 - Attributes: Other Python objects stored "inside" the object
 - One attribute that all objects have is the their type (so we can know what they are!)
 - Length of a list, number of rows in a dataframe, etc.
 - Methods: Functions associated with an object that we can use to directly access and manipulate the object's content (a function that computes the number of characters in a string variable)

Objects, attributes and methods

- In our data analysis we will import data into python objects

- Usually we want to discover some characteristics of the dataset and manipulate the data
- To do so we can use the attributes and methods of the object we imported the dataset into
- First step is to know attributes and methods of the fundamental data types

Functions

- Functions are distinguished by parentheses
 - Example: `round(number, ndigits)`
 - Can have zero or more inputs in form of arguments
 - Usually have one (possibly composite) output in form of return values
- Supports both positional and keyword arguments
 - Positional arguments: Specified by their order
 - Keyword arguments: Identified by the variable name
 - Example: `round(3.14, 1)` and `round(ndigits=1, number =3.14)` have the same output
 - Some parameters are set as default
 - Allows calling a function with fewer arguments for recurring operations
 - But also to modify the behaviour if needed

In [2]: `?round`

Methods and Imports

- Objects have attached functions called methods
 - Distinguished by a period and parentheses
 - Example: `obj.some_method(x, y, z)`
- Very conveniently let you know some (of the most commonly used) operations that can be performed on an object

Imports: Modules, packages and libraries

- Module: A single Python file (.py) containing reusable code, functions, or classes. Designed for modularity and ease of reuse
- Package: A directory containing multiple modules, organized for specific functionality. Can be made in to a library
- Library: A collection of related modules and packages, pre-compiled for specific tasks. Makes it easy to disseminate packages

Example of a Module

```
# some_module.py
PI = 3.14159

def f(x):
    return x + 2

def g(a, b):
    return a + b
```

Accessing Modules, Packages and Libraries

If you want to access all the content of a script (for example `some_module.py`) that is stored in the same directory of the notebook you are running:

```
import some_module
result = some_module.f(5)
pi = some_module.PI
```

Or if you just want some of the content:

```
from some_module import g, PI
result = g(5, PI)
```

Accessing Modules, Packages and Libraries

By using the `as` keyword, you can give imports different variable names:

```
import pandas as pd
pd.dataframe()
from some_module import PI as pi, g as gf
r1 = sm.f(pi)
r2 = gf(6, pi)
```

Exploring Methods and Imports with Tab Completion

- You can use Tab key (for autocompletion) to view what can be done with an object
 - Explores of the contents of objects, libraries, etc.
 - In the examples that follow, `<TAB>` indicates pressing the Tab key

Tab completion

Let's say that we want to know attributes and methods of a list

```
In [3]: L = [1, 2, 3]
```

We can look at the attributes and methods linked to the object using the function `dir()`.

```
In [4]: dir(L)
```

```
Out[4]: ['__add__',
        '__class__',
        '__class_getitem__',
        '__contains__',
        '__delattr__',
        '__delitem__',
        '__dir__',
        '__doc__',
        '__eq__',
        '__format__',
        '__ge__',
        '__getattr__',
        '__getitem__',
        '__gt__',
        '__hash__',
        '__iadd__',
        '__imul__',
        '__init__',
        '__init_subclass__',
        '__iter__',
        '__le__',
        '__len__',
        '__lt__',
        '__mul__',
        '__ne__',
        '__new__',
        '__reduce__',
        '__reduce_ex__',
        '__repr__',
        '__reversed__',
        '__rmul__',
        '__setattr__',
        '__setitem__',
        '__sizeof__',
        '__str__',
        '__subclasshook__',
        'append',
        'clear',
        'copy',
        'count',
        'extend',
        'index',
        'insert',
        'pop',
        'remove',
        'reverse',
        'sort']
```

Internal/External Attributes/Methods

Python does not strictly distinguish **public/external** and **private/internal** attributes

- By convention a preceding `_` is used to denote the **private/internal** (not intended to be used by the user)

- Most of these are Python's special double-underscore methods (often nicknamed "dunder" methods)
- **Private/Internal** are omitted by default, but you can list them by explicitly typing the underscore:

```
In [10]: L._<TAB>
          __add__          __delattr__      __eq__
          __class__        __delitem__      __format__()
          __class_getitem__() __dir__()      __ge__
          __contains__      __doc__          __getattr__
```

Here we are just showing the first few columns of the output

Tab Completion for Methods

```
L.<Tab>
append()  count()  insert()  reverse()
clear()   extend() pop()    sort()
copy()    index()  remove()
```

Explore their functionalities with `?`

- Tab completion works beyond the interactive namespace,
 - File and directory names
 - Functions and keywords
 - Object methods and attributes
 - Almost anything else

Tab completion - Search by First Characters

Some objects have very long lists of attributes

- You can **type the first characters of the attribute you are interested in**, the Tab key will find the matching results:

```
In [10]: dir(L) L.<TAB> L.c<TAB>
          clear() count()
          copy()
```

```
In [10]: L.co<TAB>
          copy()  count()
```

If there is only a single option, pressing the Tab key will complete the line for you.

For example, the following will instantly be replaced with `L.count` :

```
In [10]: L.cou<TAB>
```

Tab Completion When Importing

Tab completion is also useful when importing objects from packages
Here we use it to find all possible imports (all the objects) in the `itertools` package that start with `co` :

```
In [10]: from itertools import co<TAB>
          combinations()          compress()
          combinations_with_replacement() count()
```

Tab Completion When Importing

You can also use tab-completion to see the imports available on your system

- This will change depending on which third-party scripts and modules are visible to your Python session:

```
In [10]: import <TAB>
          abc          anyio
          activate_this appdirs
          aifc          appnope
          antigravity   argon2
```

```
In [10]: import h<TAB>
          hashlib html
          heapq    http
          hmac
```

Useful Python Modules

<https://docs.python.org/3/library/>

- `re` – Regular expression operations
- `datetime` – Basic date and time types
- `math` – Mathematical functions
- `random` – Generate pseudo-random numbers
- `os.path` – Common pathname manipulations
- `pickle` – Python object serialization
- `csv` – CSV file reading and writing
- `json` – JSON encoder and decoder
- ...

Useful Python Packages

- `numpy` – Scientific computing with multi-dimensional arrays
- `pandas` – Data analysis with table-like structures (R, pretty much)
- `statsmodels` – Statistical data analysis with linear models
- `scikit-learn` – Data mining and machine learning

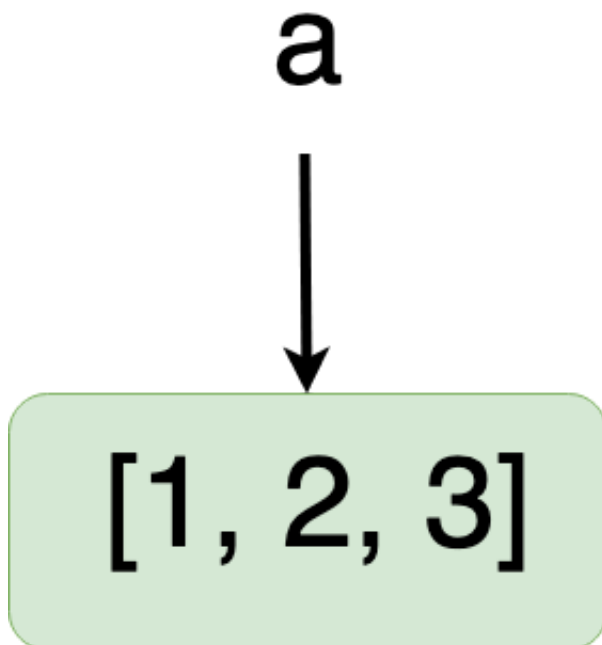
- `networkx` – Network analysis
- `matplotlib` – Plotting
- ...

Assignment and Namespaces

Assignment

When programming you will need to store information inside **variables** (Tom's boxes of lecture 1, the memory locations)

- **Assignment** of a name to an object (binding) is done using `=`
 - Creates a binding between a name and an object
 - Example: `a = [1, 2, 3]`



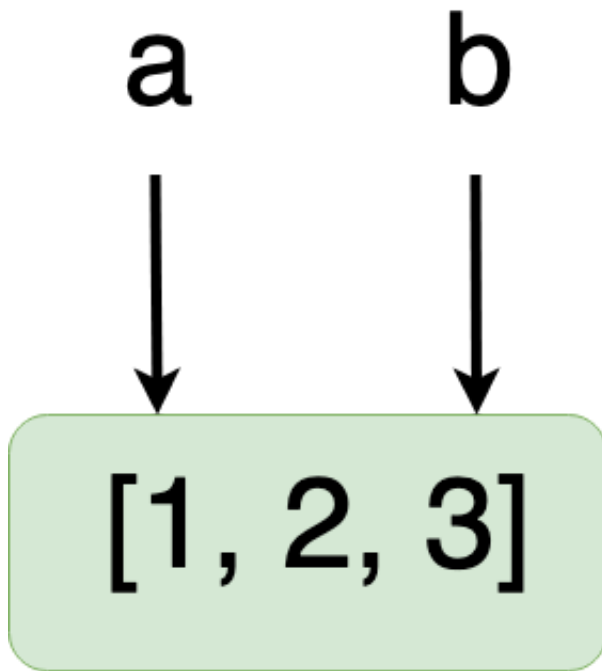
Aliasing

- If you assign a new name to an existing variable, it does not create a clone of the data
 - It binds a new name to the same object, that now has an **alias**
 - If `a` and `b` are assigned to the same object, we say that the object is **aliased** (it has two different names)

Aliasing

```
In [5]: a = [1, 2, 3]           # The name a is assigned to the list object
        b = a                   # The name b is assigned to the same list object
        print(f"a is: {a} and b is: {b}") # this is an f-string, more on that when t

a is: [1, 2, 3] and b is: [1, 2, 3]
```

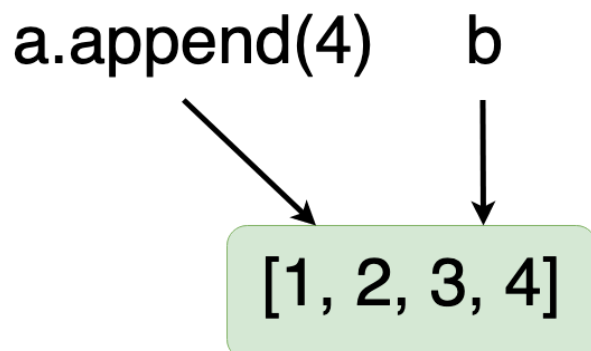


Modifying Aliased Objects

Modifications made to an object (using one of the aliases) are going to be reflected on all the aliases

```
In [6]: a = [1, 2, 3]           # The name a is assigned to the list object
        b = a                   # The name b is assigned to the same list object
        print(f"a is: {a} and b is: {b}")
        a.append(4)              # This *modifies* the list object referenced by a and b
        print(f"a is: {a} and b is: {b}")

a is: [1, 2, 3] and b is: [1, 2, 3]
a is: [1, 2, 3, 4] and b is: [1, 2, 3, 4]
```



Assignment: Modifications vs Re-Assignment

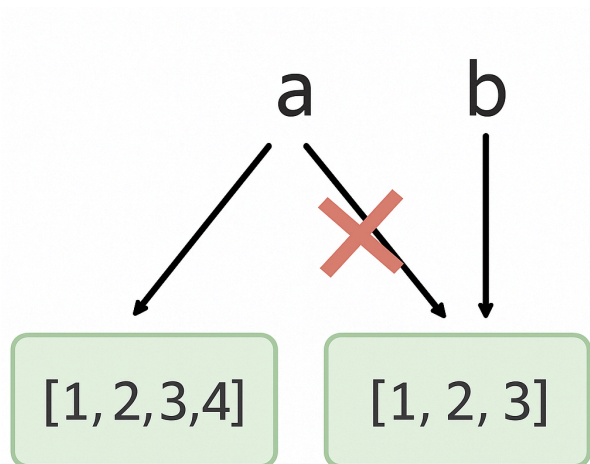
- There are crucial differences between **modifying** an existing object
 - Such as the list appending in the previous slide `a.append(4)`
- And **re-assigning** a variable
 - Such as re-assigning `a`, i.e., `a = an_object`
- **Modification** changes the underlying object
 - Affecting all the variables referring to that object
- **Re-assignment** changes the variable-object binding of one of the variable names
 - This has no impact on other variables (b is unaffected)

Assignment: Modifications vs Re-Assignment

- In the example below, re-assignment removes the initial binding and creates a new one
- This does not affect `b`

```
In [7]: a = [1, 2, 3]           # The name a is assigned to the list object
        b = a                 # The name b is assigned to the same list object
        print(f"a is: {a} and b is: {b}")
        a = [1, 2, 3, 4]      # This *reassigns* a to the new object [1, 2, 3, 4]
        print(f"a is: {a} and b is: {b}") # b is not affected
```

```
a is: [1, 2, 3] and b is: [1, 2, 3]
a is: [1, 2, 3, 4] and b is: [1, 2, 3]
```



Cloning

- If you want to do an **independent clone** of an object, creating a binding (done with the assignment operator `=`) it's not enough

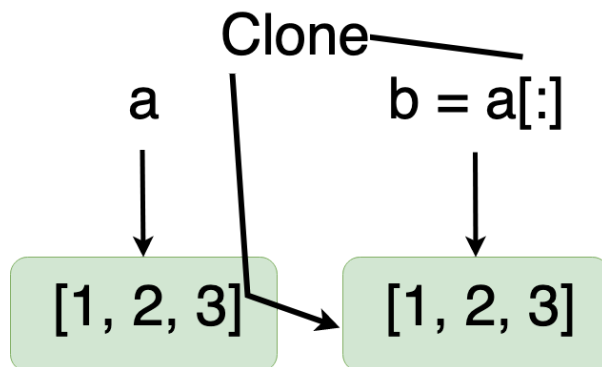
Cloning

- The simplest method to clone a list in Python is to utilize the "bare" slice operator `[:]` (the `[]` operators can do various things, we will see more in later lectures)
- You can also use the `copy()` function (from the `copy()` module) safer but may not be available

```
In [8]: b = a[:]      # make a clone using "bare" slice
print(f"a is: {a} and b is: {b}")

a.append(5)          # This *modifies* the list object referenced by a
print(f"a is: {a} and b is: {b}") # b is unaffected
```

a is: [1, 2, 3, 4] and b is: [1, 2, 3, 4]
a is: [1, 2, 3, 4, 5] and b is: [1, 2, 3, 4]



Equality (of content) is Different than Identity

To clarify it might be useful to introduce two operators

- `==` checks if the values of two operands are equal or not
 - Is the content of the box labeled `a` the same of the one in box `d`
- `is` checks if the two operands refer to the same object
 - Are the two labels `a` and `e` referring to exactly the same box (memory location)?

```
In [9]: a = [1, 2, 3]
d = a
e = a[:]
print(f"Are a and d equal? {a == d}")
print(f"Are a and d the same object? {a is d}")
print(f"Are a and e equal? {a == e}")
print(f"Are a and e the same object? {a is e}")
```

Are a and d equal? True
Are a and d the same object? True
Are a and e equal? True
Are a and e the same object? False

PRACTICE

Q1: What will the following program print?

```
ls1 = [1, 2, 3, 4, 5]
ls2 = ls1
ls2.append(2)
print(ls1)
```

- (A) [2, 1, 2, 3, 4, 5]
- (B) [2, 2, 2, 2, 2]
- (C) [1, 2, 3, 4, 5, 2]
- (D) 0

Names conflicts and Namespaces

- As we have seen in the case of re-assignment, a name can only bind to a single object
 - Assigning the same name to two different objects removes the name assignment from the first object

Data Modification, Assignment and Copying

- Most analyses start from one source dataset and yield multiple outputs (tables, plots, intermediate transforms) through stepwise changes.
- **Never modify the original in place.** You may need the untouched version for a later analysis
- **Never** use just `df_for_plot = df`. This only creates a reference and can cause silent changes
- Always create explicit copies with clear names, e.g., `df_for_plot = df.copy()`

```
In [10]: a = [1, 2, 3]
print(f"a is: {a}")
c = {"year" : 1955, "month" : 11, "day" : 5} # a dictionary
a = c
print(f"now a is: {a}")
```

```
a is: [1, 2, 3]
now a is: {'year': 1955, 'month': 11, 'day': 5}
```

- A data analysis leverages lots of libraries and functions

- Sometimes the same name might be used for different variables, functions, etc.
- E.g., a variable `variance` defined in your script and a variable with the same name in a library
- How can python manage such name conflicts?

Namespaces

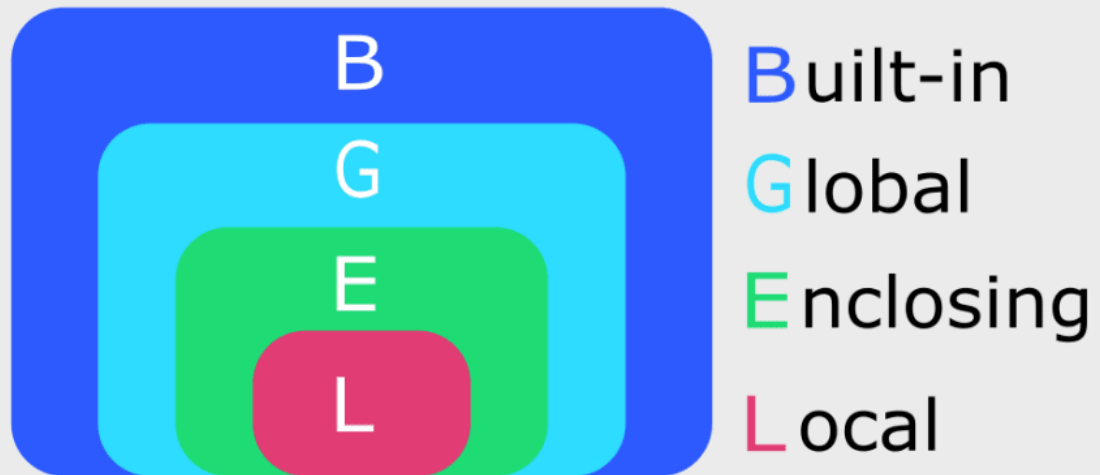
- Python implements a set of hierarchical **Namespace**
 - A set of mappings between names and objects
 - In each namespace all these names are unique and can be used without any conflict
 - Namespaces are implemented as dictionaries (names are keys, objects are values; more on dictionaries in next lectures)
- There are multiple namespaces in a Python program
 - Local: Inside a function
 - Enclosing scope: Any enclosing functions
 - Global: Top-level of the module
 - Built-in: Provided by the builtins module
- You can view the namespace with the `locals()` and `globals()` functions

Scope searching, or how to search across namespaces

- When you use a name, Python searches for it in the following order:
 - Local scope: The current function
 - Enclosing scope: Any enclosing functions
 - Global scope: The top-level of the module
 - Built-in scope: Provided by the builtins module
- Python runs the first object it finds
- Sometimes the coder messes up and the variables that is used by the code is not the intended one
 - A mistake that is very insidious and hard to identify
 - Very important to use descriptive and unique names for variables

Scope searching, or how to search across namespaces

Scopes in Python



[source](#)

Scope Searching - Global and Local x

```
In [11]: x = 1

def local_fun():
    x = 2
    print("x inside local_fun is:", x)

print("x in the global environment is:", x)
local_fun()
```

```
x in the global environment is: 1
x inside local_fun is: 2
```

[Here](#) you can find another explanation with examples

Scope creation

- What happens to the namespace when you import a module?
 - A new namespace is created for that module
 - You access the objects in that namespace using dot notation, like `module_name.object_name` (e.g., `np.array` → the function `array` in the numpy library that has been imported as `np`)
- What happens when you call a function?
 - A new local and **temporary** namespace is created for that function execution
 - This is where all the names local to the function will exist

Practical implication of scoping - Use **distinctive and informative** names

- Good practice to use distinct, informative names
 - Crucial to avoid accidental confusion of vars names
 - It will help you and others understand your code when you read it again
- Two most common naming conventions:
 - `classroom_data_2` uses the **snake case** convention
 - `parliamentSpeeches14` uses the **camel case** convention
 - Can also be combined to be more informative
 - Example `popIncome02Mar23_notClean`, `IDfirmsTextile_emilia_22`

Python Language Semantics and Structure

- Emphasis on readability, simplicity, explicitness
- Python programs are easy to write, read, and maintain
 - Often likened to "executable pseudocode"

Indentation Instead of Braces

- Python uses whitespace (indentation) to structure code
 - Tabs or spaces
- Different from languages like R, C++, Java, Perl
 - Those languages use braces
- Example: for loop that divides the elements of a vector in the elements that have values below and above a given pivot value

```
for x in vector:
    if x < pivot:
        less.append(x)
    else:
        greater.append(x)
```

Python Whitespace and Code Blocks

- A colon marks the beginning of an indented code block
- All code within the block must be indented uniformly
- Python use of whitespace might seem subtle and hard to manage but usually gets natural quickly

```

for x in vector:
    if x < pivot:
        less.append(x)
    else:
        greater.append(x)

```

Semicolons and Statement Termination in Python

- Semicolons in Python can be used to separate multiple statements on one line
 - Example: `a = 5; b = 6; c = 7`
 - Discouraged practice since it reduces code readability
- Python lines of code do not need a character for termination
 - Many other languages use semicolons `;`

Using Comments in Python

- Comments start with the hashtag (#) and are ignored by Python
 - The code after the hashtag is not run
- Comments are crucial for the readability of your code
 - You will be the one reading your code the most, make it easy on you!
- Used also for temporary code exclusion
 - Example: Commenting out code blocks for testing

```

results = []
for line in file_handle:
    ### keep the empty lines for now
    ### if len(line) == 0:
        ### continue
    results.append(line.replace("foo", "bar"))

```

Using Comments in Python

- Comments can also follow an executable line of code
- Location varies based on programmer's preference

```

### main analysis
### loop over the lines in the file
for line in file_handle:
    ### removal of empty lines
    if len(line) == 0:
        continue
    ### appending result to the list
    results.append(line.replace("foo", "bar"))
print("Reached this line") ### status report

```

Filepath and a Folder Setup for Data Analysis

How to Refer to a File in a Computer

- The hard drive is where all the computer files are stored
- How can the computer find a specific file?
 - To find something usually we need a position or an address
 - The file path is the address of a file
- An address in a computer has a tree structure
 - The root is the hard drive (e.g., `C:` on Windows, `/` on Mac)
 - The branches are the folders (e.g., Desktop, Documents, etc.)
 - The leaves are the files (e.g., `lecture03_basics.ipynb`)
- The parent folder of a file/folder is the one that contains it
 - A child folder is contained by the parent folder

Relative and Absolute Paths

- The working directory is the folder where the kernel is currently looking for files
 - e.g., `C:\Users\dgdi\projects_py\codepy\notebooks`
 - This is an absolute path: the full path to a file starting from the HD
- A path is instead called relative if it starts from the current working directory
 - If we are in the `codepy` folder, we can refer to the `lecture03_basics.ipynb` file as `notebooks\lecture03_basics.ipynb`

Limits of Absolute Paths

- Absolute paths are fine if you work on just one computer
 - Other computers might have different folder structure (e.g., a different user name)
 - On other computers your code will not be able to find the files it needs to run
- This problem can be solved by using relative paths and setting at the beginning of the code the working directory
- Let's see how to setup a project folder that allows for a simple implementation of this principle

A Basic Setup - Create a Project Directory

A well-organized project folder will save you a lot of time and worries

This is a minimal set of advices

1. Chose a folder where to you want to store all the scripts and data related to a particular project (***much better*** if it features backups, i.e. Dropbox, Gdrive, ...)

2. For each of your coding projects, create a subfolder with the project name (***much clearer*** if it has the same of the Anaconda environment, i.e., "codepy")

A Basic Setup - Create the Subfolders in the Project Directory

3. Store the notebooks in the main directory folder
4. Create subfolders to organize your analysis, a usual set of folders is:
 - a. A **data** folder for the input data
 - b. A **fig** folder for the analysis plots
 - c. A **doc** folder where to save your tables, papers, and presentations

A Basic Setup - Using Relative Paths

5. Usually, when you open a notebook the working directory is set to the folder that contains it (parent folder). So you can just use relative paths in your notebooks

```
import pandas as pd
GDP = pd.read_csv("data/GDP_istat.csv")
```

- This code takes as a starting point the folder where the notebook is placed (the main project directory)
- Search in the folder `data`
- Opens the file `GDP_istat.csv`
- Stores it in the variable GDP

A Basic Setup - Setting the Working Directory to the Parent of the Notebook

6. It may happen that the working directory is not set to the parent folder of the notebook. If that is the case you can use the following code to set it right

```
import os
notebook_dir = "Path/of/a/folder" #stores the path of the
#directory in the variable notebook_dir
os.chdir(notebook_dir)             #set the working directory to
#the notebook_dir value

# Check the new working directory
print("Current working directory:", os.getcwd())
```

You can easily get the "Path/of/a/folder"

- On Mac, right-click on the folder of interest while pressing the button option and select "copy -name of the folder- as pathname"

- On Win, right-click on the folder of interest and select "copy as path"

Finding Information is Key to Coding

Finding information is key to coding

- In and Data Analysis most of the time it's less a matter of knowing everything needed to do something and more about knowing how to quickly discover it
- A wealth of information is available through searchable web resources such as, [online documentation](#), [Stack Overflow](#), [Github issue](#), AI assistants are another great resource to ask questions and find answers
 - Better for simple task, great for beginners
- Effective data scientists do not memorize commands for every possible situation (**autocompletion**)
 - More about learning to effectively find the information you don't know
 - Usually a web search will reveal the solution
 - Usually you will search multiple times for the solution to a problem you've already solved before
 - The more you practice, the more you'll remember the solution to common problems

Code documentation

- Code documentation is a set of documents that describe the functionality of a code
 - Partly available in the code itself
 - Partly available in external documents online
- It is usually written by the developers of the code
- Usually very structured and abstract (but recently more examples tend to be included)
 - At the beginning might be difficult to read
 - Once you get used to it, you'll see that structure and abstraction improve clarity
- The first source of information you should look out for
 - It collects the main info you need to know about a specific tool, library, function
 - It is the most reliable source of information
 - Sometimes not immediate to find the info you need
 - Sometimes very detailed, sometimes not enough

Crowdsourced Knowledge - Stack Overflow

- Stack Overflow is a question and answer site for programmers

- It's part of the Stack Exchange network of Q&A sites (other sites include MathOverflow, stats.stackexchange, etc.)
- It's a great resource for finding answers to specific questions
 - It's likely that someone else has already faced the same problem you are facing
 - It's likely that someone else has already solved the problem
- It's also a great resource for asking your specific question
 - But make sure you follow the guidelines for asking a good question
 - These guidelines are essential to the effectiveness of the platform and **enforced** by the community
 - Only ask for the time of others after you can show that you care (effort is the currency for getting attention)

Crowdsourced Knowledge - Stack Overflow

- The risk on Stack Overflow is that you might get lost in the flood of information
 - Or that you use a solution that you are not really understanding
 - Or that you use a solution that is not really solving your problem

Crowdsourced Knowledge - Github Issues

- Github is a platform for hosting and collaborating on code
 - It's the most popular platform for open source code
 - It's also used for private code repositories
- Github issues are a way to track bugs, feature requests and to ask questions on specific aspects of the code
 - It's a great resource for finding answers to specific questions
 - The level of detail on average is higher than Stack Overflow
 - You are in direct contact with the developers of the code and the community of users
 - Which is great but comes with the responsibility of asking good questions
- Same cons of Stack Overflow

Google is Your Friend

- Google is the most immediate tool for finding information online
 - Allows you to search on the resources mentioned above plus a lot more (tutorial, blogs, etc.)
- Searching for code related information is a skill that you need to develop
 - It's not just about typing the right keywords
 - It's about understanding the results and finding the right information

- It's about understanding the quality of the information you find
- The more you practice, the better you'll get at it



Python Integrated Help and Documentation

- Web searches play an important role in solving complicated questions,
- However, an amazing amount of information can be found through IPython alone
 - Usually what you need in simple cases
 - **The only source of information that will be available during the exam**
- Some examples of the questions IPython can help answer in a few keystrokes include:
 - How do I call this function? What arguments and options does it have?
 - What does the source code of this Python object look like?
 - What is in this package I imported?
 - What attributes or methods does this object have?
- The functions and methods you use in Python have a built-in documentation
- One of the most useful features of IPython/Jupyter is readily provide documentation and information search
 - You can access it using the `?` character or the `help()` function
 - You can explore the source code of function using the `??`

Accessing Documentation with `?`

- Documentation for each object is accessible through *docstrings*:
 - Concise summary of the object and how to use it

- Python's `help` function displays this information
- For Example let's consider the help for the `len` function

In [12]: `?len`

- Use `?` for quick access to this documentation and other relevant information

In [13]: `len?`

Accessing Source Code with ??

- Python's readability allows easy understanding of source code.
 - Use `??` in IPython/Jupyter for quick access to source code.

```
In [14]: ###defining a function that squares a number
def square(a):
    """Return the square of a."""
    return a ** a
square??
```

- Docstring can be created along with the function
 - Place a string literal at the beginning
 - Essential for code reusability and understanding
 - Benefitting both **future** you and other developers

Accessing Source Code with ?? - Limitations

- `??` may not show source code if object is implemented in C or another compiled language.
- In such cases, `??` yields same output as `?`.
- Common with Python's built-in objects and functions, like `len`.

In [15]: `len?`

In [16]: `len??`

Exceptions and Errors

What is Coding (Mostly) About?

99 little bugs in the code,

99 bugs in the code,

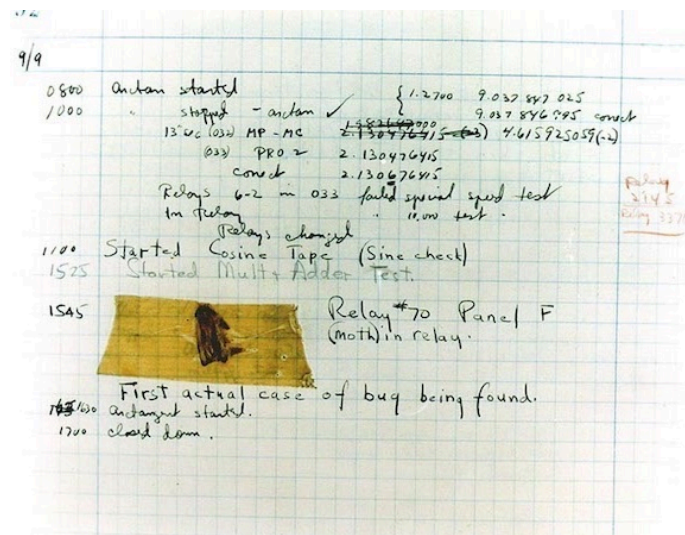
1 bug fixed...run again,

100 little bugs in the code...

Coding and data analysis is a continuous trial and error

- You are going to do multiple mistake for each operation you want to do
 - Very stressful at first, be committed and patient with yourself
- We will now briefly talk about how to approach de-bugging and the available options to deal with exceptions (errors, warnings, etc.)

Computer Bugs



The actual first computer bug. On September 9, 1947, Admiral Grace Hopper found this moth trapped on a relay of the Harvard Mark II computer. (Image source: U.S. Naval Historical Center Online Library)

How to Debug:

1. Try Avoiding bugs in the first place
 - Start with very simple statments
 - Always print intermediate results
 - Very often what you think should be the output and the real output are different (**90% of beginners errors are here!**)
 - Use descriptive names for variables
2. External resources/Learn from Other Programmers: Use **Google/Stackoverflow/Github/AI** to better understand and find a solution to your problem

Debugging Systematically

1. Use `print()` to investigate what is inside your variables at crucial steps
 - Compare input in successful and failing runs - **START SIMPLE!!!**
 - Formulate a hypothesis on the location of the problem
 - Design an experiment and test the hypothesis using `print()`
 - Keep record of your experiment (so you can re-check in case)
 - Repeat

Getting Info From The Error

When a Python script fails, usually it will raise an exception

- The console will display the *traceback* with the information on the cause of the exception
 - Detailing the chain of function calls leading to the error
 - Somewhere in that chain there is the problem
- Use `%xmode` (*exception mode*) to control the amount of information printed when the exception is raised

Before seeing a simple example, let's introduce the family of `%xmode` magic commands:

IPython Magic Commands

- IPython entails *magic commands* that are prefixed by the `%` character
- Magic commands succinctly solve various common problems in data analysis, they come in two flavors:
 - *line magics*, prefixed by a single `%`, that operate on a single line of input
 - *cell magics*, prefixed by a double `%%`, that operate on multiple lines of inputs
- There are many, we only review a couple to get an idea

Two functions

Here we define two functions:

- `func1` that takes one argument `x`, creates the variable `a = x` and the variable `b = x - 1` and then applies `func2`
- `func2` takes two arguments `a` and `b` and computes `a/b`

```
In [17]: def func1(x):  
         a = x  
         b = x - 1  
         return func2(a, b)  
  
def func2(a, b):  
    return a / b
```

Controlling Exceptions: %xmode

```
In [18]: func1(1)
```

```
-----  
ZeroDivisionError                                Traceback (most recent call last)  
Cell In[18], line 1  
----> 1 func1(1)  
  
Cell In[17], line 4, in func1(x)  
      2 a = x  
      3 b = x - 1  
----> 4 return func2(a, b)  
  
Cell In[17], line 7, in func2(a, b)  
      6 def func2(a, b):  
----> 7     return a / b  
  
ZeroDivisionError: division by zero
```

Default Traceback

- Calling `func2` results in an error, coming from its 7th line, coming from `func1`, during the division at line 2
- By default (`Context`), traceback shows each step leading to the error and the lines where the exception is raised
- `%xmode` takes one of three possibilities: `Plain`, `Context`, and `Verbose`.
 - The default is `Context`
 - `Plain` is more compact and gives less information:
 - The `Verbose` mode adds some extra information, including the arguments to any functions

```
In [19]: %xmode Plain  
func1(1)
```

```
Exception reporting mode: Plain  
Traceback (most recent call last):  
  
Cell In[19], line 2  
    func1(1)  
  
Cell In[17], line 4 in func1  
    return func2(a, b)  
  
Cell In[17], line 7 in func2  
    return a / b  
  
ZeroDivisionError: division by zero
```

With `Plain` we no longer see the code where the exception is raised


```
In [20]: %xmode Verbose
func1(1)
```

Exception reporting mode: Verbose

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[20], line 2
      1 get_ipython().run_line_magic('xmode', 'Verbose')
----> 2 func1(1)

Cell In[17], line 4, in func1(x=1)
      2 a = x
      3 b = x - 1
----> 4 return func2(a, b)
      a = 1
      b = 0

Cell In[17], line 7, in func2(a=1, b=0)
      6 def func2(a, b):
----> 7     return a / b
      a = 1
      b = 0

ZeroDivisionError: division by zero
```

Using the `Verbose` option we also have the information on the values of variables

Error information

The extra information can help you understand why the exception is being raised

Which traceback option is best depends on the complexity of the code you are debugging

- `Verbose` provides a lot of information but the traceback can get extremely long
- Depending on the context, the brevity of `Plain` or `Context` may be easier to work with.

Debugging: When Reading Tracebacks Is Not Enough

The IPython-enhanced version of this is `ipdb`, the IPython debugger.

- This debugger lets you step through the code line by line
- Many ways to launch/use the debuggers
 - The most convenient interface to debugging is the `%debug` magic command
 - Refer to the online documentation to learn more
- If you run `%debug` after an exception is raised, it will open an interactive debugging prompt at the point of the exception.
- The `ipdb` prompt lets you explore the current state of the stack (the "pile" of commands that you have sent to be executed), explore the available variables, and

even run Python commands!

- Out of the scope of this course but a powerful tool that you need to know exists

After Debugging for Hours...

- Stop
- Try commenting your code or explaining it to someone else
- Sleep on it



(Image source: Reddit)

Debugging with AI

- What AI can do
 - Explain error messages and traceback in plain language
 - Suggest likely causes and concrete fixes (code patches, parameter changes)
 - Propose debugging steps (print checks, assert statements, logging)
- Best practices when asking an AI
 - Provide your code and the error output
 - Ask for multiple hypotheses and a prioritized list of fixes

Debugging with AI

- Limitations & safety
 - Validate any AI-suggested change locally and with tests — do not apply blindly
 - Be careful sharing sensitive data (private data, login information) in prompts
 - Combine AI suggestions with human review and automated tests
- Workflow once you get an error
 1. copy the input and output (error message, traceback)
 2. Ask AI for explanations, hypotheses, and patches
 3. If you do not like the answer, generate a minimal reproducible example to put on SO
 4. Apply changes incrementally and run tests
 5. Review and document the fix

Homework

- Set up the project folder, including the subfolders

Notes

Book references:

Ch.2 of [Python for Data Analysis](#)

Ch.1 of [Python Data Science Handbook](#)

Office Hours:

See on my [website](#)

Author: Duccio Gamannossi degli Innocenti

Web: <https://www.dgdi.me/>

Mail: duccio.gamannossi@unipd.it

